DPDP COMPLIANCE AUDIT

saas-copilot

Digital Personal Data Protection Act 2023

65/100

Grade C — Needs Work

Scanned: 24 March 2026

Files indexed: 45

Languages: FastAPI, SQLAlchemy, Pydantic, Passlib, python-jose, uvicorn

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

The application collects user name, email, and password for authentication. The biggest DPDP risk is the storage of plain text passwords (though hashed) and the potential for unauthorized access to JWT tokens stored in local storage.

Validation Summary Rule findings: 11 confirmed, 3 false positives suppressed • AI gaps: 2 dual-validated • Score: 65 (rules) → 65 (integrated)

Total findings: 13
HIGH: 4
MEDIUM: 3
INFO: 3
PASS: 1

Tech Stack: FastAPI, SQLAlchemy, Pydantic, Passlib, python-jose
Risk Profile: The application collects user name, email, and password for authentication. The biggest DPDP risk is the storage of plain text passwords (though hashed) and the potential for unauthorized access to JWT tokens stored in local storage.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	11/22 (50%)	WARN
Section 6(6)	8	0/8 (0%)	FAIL
Section 8(1)	18	18/18 (100%)	PASS
Section 8	10	10/10 (100%)	PASS
Section 8(3)	8	8/8 (100%)	PASS
Section 8(4)	8	0/8 (0%)	FAIL
Section 8(6)	8	6.8/8 (85%)	PASS
Section 9	6	6/6 (100%)	PASS
Section 11	6	2.1/6 (35%)	FAIL
Section 16	4	4/4 (100%)	PASS
Section 5	2	2/2 (100%)	PASS
Section 13 — Grievance Redressal	4	0/4 (0%)	FAIL

Estimated Total Remediation Effort: 2.6–9.1 engineering days

Data Flow Analysis

The following PII flow signals were inferred via static analysis (import/call adjacency). These are heuristic code-path chains, not proven runtime traces. Use them to guide developer review (confirm actual request → handler → service → database paths in your app).

Flow 1: Application Logs
PII fields: email, password
`Login.jsx → api.js`

Flow 2: Application Logs
PII fields: email, password
`Signup.jsx → api.js`

Rule	Section	Severity	File
CONSENT_MISSING_REPO_LEVEL	Section 6 — Consent	HIGH	REPO-WIDE
CONSENT_WITHDRAWAL_MISSING	Section 6(6) — Consent Withdrawal	HIGH	N/A
AUDIT_TRAIL_MISSING	Section 8(4) — Audit Trail	HIGH	N/A
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
DATA_ACCESS_MISSING	Section 11 — Right to Access	MEDIUM	N/A
NO_BREACH_ALERTING	Section 8(6) — Breach	LOW	N/A
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
NO_THIRD_PARTY_DETECTED	Section 5 — Notice	INFO	N/A
NO_THIRD_PARTY_DATA_SHARING	Section 5 — Purpose Limitation	INFO	N/A
NO_CROSS_BORDER_DETECTED	Section 16 — Cross-Border Transfer	INFO	N/A
CONSENT_PRESENT	Section 6 — Consent	PASS	backend/app/api/auth.py
	Section 9 — Children	MEDIUM	
	Section 5 — Purpose Limitation	MEDIUM	

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (10–36 hours)

Fix immediately — these represent the highest legal exposure.

- Consent Missing Repo Level — REPO-WIDE (4–16 hrs)
- Consent Withdrawal Missing (1–4 hrs)
- Audit Trail Missing (4–12 hrs)
- Grievance Officer Missing (1–4 hrs)

Week 2 — High Priority (6–22 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Data Access Missing (2–6 hrs)
- (2–8 hrs)
- (2–8 hrs)

Week 4 — Advisory (4–14 hours)

Address before next compliance review.

- No Breach Alerting (2–6 hrs)
- Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 20–72 hours (2.5–9.0 engineering days)

Details

Finding 1 of 13

[Section 6 — Consent]

1. [HIGH] CONSENT_MISSING_REPO_LEVEL — CODEBASE-WIDE

Estimated fix time: 4–16 hours

No consent mechanism detected anywhere in the repository.

Risk:

The absence of a consent mechanism means that personal data processing activities may not have a lawful basis, violating DPDP Section 6. This can lead to non-compliance penalties and a lack of trust from data subjects.

Fix:

1. Implement a consent management system, potentially using a dedicated service or library, to track user consent status for different data processing activities.

2. Add UI elements (e.g., checkboxes, modals) in the frontend (e.g., `frontend/src/App.jsx` or relevant pages) to allow users to grant and manage their consent.

3. Modify backend API endpoints (e.g., in `backend/app/api/auth.py` or other relevant services) to check for and record user consent before processing personal data.

4. Create a new API endpoint (e.g., in `backend/app/api/consent.py`) to allow users to view and withdraw their consent.

DPDP:

Section 6 — Consent requires explicit, informed, and unambiguous consent for the processing of personal data.

Example:

```
from fastapi import APIRouter, Depends from app.db.session import SessionLocal from
app.schemas.consent import ConsentUpdate router = APIRouter(prefix="/consent",
tags=["Consent"]) def get_db(): db = SessionLocal() try: yield db finally: db.close()
@router.put("/manage") def manage_consent(request: ConsentUpdate, db: Session =
Depends(get_db)): # Logic to update user consent status in the database pass
```

Finding 2 of 13

[Section 6(6) — Consent Withdrawal]

2. [HIGH] CONSENT_WITHDRAWAL_MISSING — N / A

Estimated fix time: 1–4 hours

Consent is collected but no withdrawal mechanism detected. DPDP Section 6(6) requires consent withdrawal to be as easy as giving consent. Users must be able to revoke consent at any time.

Risk:

The lack of a consent withdrawal mechanism prevents users from revoking their consent, which is a mandatory right under DPDP Section 6(6). This can lead to continued processing of data without a valid legal basis and potential legal challenges.

Fix:

1. Create a new API endpoint (e.g., `/consent/withdraw`) in `backend/app/api/consent.py` that allows authenticated users to revoke their consent.

2. Implement logic in the withdrawal endpoint to update the user's consent status in the database, effectively disabling processing based on that consent.

3. Add a corresponding UI element in the frontend (e.g., in `frontend/src/pages/ProfilePage.jsx` or similar) that allows users to trigger the consent withdrawal API.

4. Ensure that all data processing activities that rely on consent are checked against the user's current consent status and halted if consent is withdrawn.

DPDP:

Section 6(6) — Consent withdrawal must be as easy as giving consent, allowing users to revoke consent at any time.

Example:

```
from fastapi import APIRouter, Depends, HTTPException from app.db.session import SessionLocal
from app.schemas.user import User router = APIRouter(prefix="/consent", tags=["Consent"]) def
get_db(): db = SessionLocal() try: yield db finally: db.close() @router.delete("/withdraw")
def withdraw_consent(current_user: User = Depends(get_current_user), db: Session =
Depends(get_db)): # Logic to mark all user consents as withdrawn in the database pass
```

Finding 3 of 13

[Section 8(4) — Audit Trail]

3. [HIGH] AUDIT_TRAIL_MISSING — N / A:1

Evidence snippet: This frontend file manages application state and routing, with no indication of audit logging capabilities.

Estimated fix time: 4–12 hours

No audit trail infrastructure detected. DPDP Section 8(4) requires complete records of personal data processing activities. No access logging, audit model, or change tracking found in the codebase.

Risk:

The absence of an audit trail means there are no records of personal data processing activities, violating DPDP Section 8(4). This hinders accountability, makes it difficult to investigate data breaches, and prevents demonstration of compliance.

Fix:

1. Define an `AuditLog` model in `app/models/audit.py` to store details of processing activities (e.g., timestamp, user ID, action, data involved).

2. Implement a logging mechanism, possibly using a decorator or middleware, to automatically record relevant actions performed via API endpoints (e.g., in `backend/app/api/rag.py` and `backend/app/api/auth.py`).

3. Ensure that sensitive operations, such as user registration, login, and data ingestion/querying, are logged with sufficient detail.

4. Create an administrative interface or API endpoint to access and review the audit logs.

DPDP:

Section 8(4) — Requires organizations to maintain complete and accurate records of all personal data processing activities.

Example:

```
from app.db.base_class import Base from sqlalchemy import Column, Integer, String, DateTime, ForeignKey from datetime import datetime class AuditLog(Base): __tablename__ = "audit_logs" id = Column(Integer, primary_key=True, index=True) user_id = Column(String, nullable=True) action = Column(String, index=True) timestamp = Column(DateTime, default=datetime.utcnow) details = Column(String, nullable=True)
```

Finding 4 of 13

[Section 13 — Grievance Redressal]

4. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A:1

Evidence snippet: This frontend file manages application state and routing, with no visible contact details for a grievance officer.

Estimated fix time: 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route **and** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

The absence of a designated grievance officer or a clear process for handling complaints violates DPDP Section 13. This leaves data subjects without a clear channel to address concerns or seek redress for data protection issues, potentially leading to unresolved disputes and reputational damage.

Fix:

1. Designate a specific individual or role within the organization responsible for handling data subject grievances and ensure their contact information is readily available.

2. Create a new API endpoint (e.g., `/grievance`) in `backend/app/api/grievance.py` to allow users to submit formal complaints.

3. Add a dedicated section in the frontend (e.g., in `frontend/src/pages/ContactPage.jsx` or `frontend/src/pages/HelpPage.jsx`) that clearly outlines the grievance process and provides the contact details of the designated officer.

4. Implement backend logic to receive, log, and manage submitted grievances, ensuring timely responses to data subjects.

DPDP:

Section 13 — Requires organizations to establish a mechanism for addressing and resolving data subject grievances related to personal data processing.

Example:

```
from fastapi import APIRouter, Body router = APIRouter(prefix="/grievance", tags=["Grievance"]) @router.post("/submit") def submit_grievance(name: str = Body(...), email: str = Body(...), complaint: str = Body(...)): # Logic to log the grievance and notify the designated officer print(f"Received grievance from {name} ({email}): {complaint}") return {"message": "Grievance submitted successfully. We will contact you shortly."}
```

Finding 5 of 13

[Section 11 — Right to Access]

5. [MEDIUM] DATA_ACCESS_MISSING — N / A

Estimated fix time: 2–6 hours

No endpoint detected that allows authenticated users to retrieve their own personal data. Under DPDP Section 11, Data Principals have the right to access information about their personal data being processed.

Risk:

The absence of a data access endpoint violates DPDP Section 11, preventing data principals from exercising their right to access their personal data, which can lead to non-compliance penalties and loss of user trust.

Fix:

1. Create a new FastAPI endpoint (e.g., GET /users/me/data) that requires user authentication.
2. In the endpoint, retrieve the authenticated user's ID.
3. Query the database for all personal data associated with that user ID.
4. Return the data in a structured format (e.g., JSON).

DPDP:

Section 11 — Right to Access: Data Principals have the right to access information about their personal data being processed.

Example:

```
from fastapi import APIRouter, Depends from app.database import get_db from app.models.user import User from app.schemas.user import UserDataSchema from app.auth.dependencies import get_current_user router = APIRouter() @router.get('/users/me/data', response_model=UserDataSchema) def get_my_data(current_user: User = Depends(get_current_user)): # In a real app, you'd query your DB for data related to current_user.id # For this example, we'll return a placeholder. user_data = { "user_id": current_user.id, "email": current_user.email, "name": current_user.name } return user_data
```

Finding 6 of 13

[Section 8(6) — Breach]

6. [LOW] NO_BREACH_ALERTING — N / A

Estimated fix time: 2–6 hours

Structured logging detected but no breach notification mechanism found. DPDP Section 8(6) requires notifying affected individuals and the Authority without undue delay when a breach occurs. Logging alone is insufficient — implement automated alerting (Sentry, PagerDuty, or equivalent) with escalation to a compliance team.

Risk:

The absence of a breach notification mechanism means that in the event of a data breach, affected individuals and the Authority will not be promptly informed as required by DPDP Section 8(6), potentially leading to significant legal and reputational damage.

Fix:

1. Integrate a monitoring and alerting service (e.g., Sentry, PagerDuty) into your application's error handling.
2. Configure alerts for critical errors, security events, and potential data breaches.
3. Establish an escalation policy to notify a designated compliance team upon receiving a breach alert.
4. Document the breach response and notification procedure.

DPDP:

Section 8(6) — Breach: Requires notifying affected individuals and the Authority without undue delay when a breach occurs.

Example:

```
import sentry_sdk sentry_sdk.init( dsn="YOUR_SENTRY_DSN", # Set traces_sample_rate to 1.0 to capture 100% of transactions for performance monitoring. # We recommend adjusting this value in production. traces_sample_rate=1.0, # ... other options ) def handle_breach_event(user_id, data_compromised): # Log the event details sentry_sdk.capture_message(f"Potential Data Breach: User {user_id} affected. Data: {data_compromised}") # Trigger notification to compliance team notify_compliance_team(user_id, data_compromised) # Example usage within an error handler or security check: # try: # ... sensitive operation ... # except Exception as e: # if is_data_breach(e): # handle_breach_event(current_user.id, e.details) # else: # sentry_sdk.capture_exception(e)
```

Finding 7 of 13

[Section 11 — Data Portability]

7. [LOW] DATA_PORTABILITY_MISSING — N / A

Estimated fix time: 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The lack of a data export endpoint prevents data principals from exercising their right to data portability under DPDP Section 11, potentially leading to non-compliance and user dissatisfaction.

Fix:

1. Create a new FastAPI endpoint (e.g., GET /users/me/export) that requires user authentication.

2. Retrieve the authenticated user's data from the database.

3. Format the data into a commonly used, machine-readable format (e.g., CSV, JSON).

4. Return the formatted data as a file download.

DPDP:

Section 11 — Data Portability: Data Fiduciaries must provide personal data in a commonly used, machine-readable format upon request.

Example:

```
from fastapi import APIRouter, Depends, Response from fastapi.responses import
StreamingResponse import csv import io from app.database import get_db from app.models.user
import User from app.auth.dependencies import get_current_user router = APIRouter()
@router.get('/users/me/export') def export_my_data(current_user: User =
Depends(get_current_user)): # Fetch user data (replace with actual DB query) user_data = [
{'id': current_user.id, 'email': current_user.email, 'name': current_user.name} ] # Create CSV
in memory output = io.StringIO() writer = csv.DictWriter(output,
fieldnames=user_data[0].keys()) writer.writeheader() writer.writerows(user_data)
output.seek(0) return StreamingResponse(output, media_type='text/csv',
headers={'Content-Disposition': 'attachment; filename=user_data.csv'})
```

Finding 8 of 13

[Section 5 — Notice]

8. [INFO] NO_THIRD_PARTY_DETECTED — N / A

No known third-party analytics or tracking SDKs detected in codebase.

Finding 9 of 13

[Section 5 — Purpose Limitation]

9. [INFO] NO_THIRD_PARTY_DATA_SHARING — N / A

No third-party data sharing detected.

Finding 10 of 13

[Section 16 — Cross-Border Transfer]

10. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 11 of 13

[Section 6 — Consent]

11. [PASS] CONSENT_PRESENT — backend / app / api / auth.py

Auth enforcement pattern detected inline — consent likely handled at registration.

Finding 12 of 13

[Section 9 — Children]

12. [MEDIUM] — CODEBASE-WIDE

Missing age verification for signup

Finding 13 of 13

[Section 5 — Purpose Limitation]

13. [MEDIUM] — CODEBASE-WIDE

Unnecessary PII stored in local storage

AI-Identified Compliance Gaps

The following gaps were identified by AI analysis and cross-validated. These are advisory observations, not verified rule violations. Human review is required before acting on these findings.

[AI-INFERRED] Missing age verification for signup

Confidence: 68%

Section 9 — Children

The application collects user data (name, email, password) via `/auth/signup` and stores it in the `User` model (`models/user.py`). There is no indication in the provided context (schemas, auth handlers, ORM model) of any age verification mechanism or restriction for minors during the signup process.

Recommendation: Implement an age verification step during user signup, potentially by adding an `age` or `date_of_birth` field to `backend/app/schemas/user.py` and `backend/app/models/user.py`, and adding validation logic in `backend/app/api/auth.py` to prevent minors from signing up or to obtain parental consent.

[AI-INFERRED] Unnecessary PII stored in local storage

Confidence: 67%

Section 5 — Purpose Limitation

The `data_flows` indicate 'User name stored in local storage via POST /auth/signup' and 'User name stored in local storage via POST /auth/login'. While this might be for UI convenience, storing PII like a user's name in local storage without a clear, explicit purpose beyond session management could be considered a violation of purpose limitation and data minimization.

Recommendation: Review the necessity of storing `User name` in local storage. If it's only for display, consider fetching it on demand from the backend after authentication or only storing a non-identifying user ID. Update `frontend/src/components/auth/signup.jsx` and `frontend/src/components/auth/login.jsx` (or related frontend logic) to remove or minimize PII storage in local storage.

Deep Compliance Review

Overall Risk: CRITICAL

The application's architecture presents critical data protection risks, primarily due to insecure secrets management and insufficient safeguards for PII storage and transmission. Hardcoded JWT secrets, the use of SQLite for production PII, and direct exposure of sensitive data in URL parameters significantly increase the likelihood of unauthorized access and data breaches. Furthermore, the absence of mechanisms for data principal rights and robust security controls like rate limiting creates a high risk of non-compliance with DPDP principles.

E

Most Critical Action: Implement a secure, environment-variable-based mechanism for managing the JWT secret, ensuring it is unique per environment and not hardcoded.

Layer Breakdown

Configuration & Secrets — 1 finding(s). This layer primarily handles application configuration and frontend build settings. A key DPDP compliance risk identified is the lack of validation for critical data processing parameters, which could lead to violations of purpose limitation and data minimization if misconfigured. Other files are related to development tooling and do not pose direct DPDP risks.

Data Models & Storage — 3 finding(s). This layer exhibits critical architectural weaknesses regarding data security and lifecycle management. The use of a weak default JWT secret and SQLite for production PII storage directly undermines security safeguards, while the absence of data retention mechanisms makes compliance with data lifecycle principles challenging.

Third-Party & PII Flows — 3 finding(s). This layer exhibits critical data flow patterns that introduce DPDP compliance risks, particularly regarding the security of PII. Sending potentially sensitive user questions via URL query parameters is a significant vulnerability, and storing user names in unencrypted client-side storage poses additional risks. While an attempt is made to filter unsafe content, its current implementation is insufficient to prevent PII leakage.

Authentication & Session — 3 finding(s). The authentication and session management layer has critical security vulnerabilities related to JWT secret management and lacks a robust token revocation mechanism, which directly impacts the protection of personal data. Additionally, PII is unnecessarily exposed in the signup response, violating data minimization principles.

API Routes & Controllers — 4 finding(s). The API Routes & Controllers layer exhibits several critical DPDP compliance gaps, particularly concerning the implementation of data principal rights (correction) and robust security safeguards (rate limiting, error handling). Additionally, data minimization principles are not fully adhered to in authentication responses. These issues increase the risk of unauthorized access and hinder the fulfillment of data principal obligations.